Matemáticas de la Especialidad Matemática Industrial
Prueba de evaluación progresiva 1 - Marzo de 2026

Nombre y apellidos: Número de matrícula:

INFORMACIÓN IMPORTANTE:

- **No** se permite **utilizar internet** en ningún momento a no ser que el profesor lo indique.
- **No** se permite **conectar** ningún **dispositivo** al ordenador durante el examen.
- Si se desea **entregar** el examen **antes** de la finalización del mismo, se deberá **avisar** a alguno de los profesores para subir los archivos a Moodle de forma vigilada.
- Las cuestiones de **teoría** se pueden responder **en hojas aparte**, sin límite de folios. Incluir nombre y apellidos en todas las hojas.
- Se **permite** utilizar cualquier **función** disponible en la **página del curso**. No olvidar **actualizar** el ***path*** de Matlab para que éste tenga acceso a todas las funciones.
- **No se admiten preguntas**. Si se considera que el enunciado contiene alguna ambigüedad, tómese la decisión que se considere oportuna y justifíquese en papel o dentro del propio código.
- Para facilitar la corrección, nombrar las funciones **exactamente** como se indica en el enunciado, sin añadir iniciales ni ningún otro término.
- Si los códigos entregados contienen errores de ejecución, se **puntuarán con un cero**. Por tanto, es fundamental comprobar que los códigos se pueden ejecutar desde el principio hasta el final.

Tiempo:

2h 30 min.

Problema 1 (4.5pt)

Consideremos un sistema no lineal de la forma $\mathbf{f}(\mathbf{x}) = \mathbf{0}$. Se desea implementar una modificación del método de Newton–Raphson que consiste en calcular la matriz jacobiana $\mathbb{J} := \partial\mathbf{f}/\partial\mathbf{x}$ únicamente cada K iteraciones, siendo K un dato, y mantener fijo su valor en las iteraciones intermedias. Es decir, las iteraciones $k = 0, K, 2K, \ldots$ son de la forma habitual

$$\begin{aligned} \mathbb{F}^k &:= \mathbb{J}(\mathbf{x}^k), \\ \mathbf{p}^k &:= -\left(\mathbb{F}^k\right)^{-1} \mathbf{f}^k, \\ \mathbf{x}^{k+1} &:= \mathbf{x}^k + \mathbf{p}^k, \end{aligned}$$

mientras que el resto de iteraciones reutilizan la matriz jacobiana calculada en la iteración anterior,

$$\begin{aligned} \mathbb{F}^k &:= \mathbb{F}^{k-1}, \\ \mathbf{p}^k &:= -\left(\mathbb{F}^k\right)^{-1} \mathbf{f}^k, \\ \mathbf{x}^{k+1} &:= \mathbf{x}^k + \mathbf{p}^k. \end{aligned}$$

Cuestión 1 (1pt): *Indicar las ventajas e inconvenientes que presenta el método anterior.*

En lo que sigue, se supone que $\mathbb{J}$ es una matriz llena (es decir, no dispersa) y no simétrica.

Cuestión 2 (3pt): *Implementar una función*

```
[x_k, nIters, flag] = NewtonModif(fun, x0, Tol, MaxIter, K),
```

donde los argumentos de entrada y de salida, a excepción de K, son los mismos que los de la función `NewtonRaphson1`*, que implemente el método descrito arriba de forma eficiente.*

Cuestión 3 (0.5pt): *Implementar una función* `Polin_NewtonModif(N,K)` *que, recibidos los valores de N y K, resuelva el sistema polinomial (sección 6.3 de los apuntes) utilizando el método implementado en* `NewtonModif`*. Fijando $N = 4000$, completar la tabla 1 con el número de iteraciones y los tiempos empleados según los distintos valores de K. Justificar los resultados.*

Tabla 1: Número de iteraciones y tiempos de cálculo según el valor de K.

K	Número de iteraciones	Tiempo de resolución del sistema [s]
1		
2		
5		
10		

Problema 2 (5.5pt)

Una pared de espesor $L = 0.1\,m$ separa un líquido a temperatura $T_0 = 400\,K$ de un ambiente a temperatura $T_\infty = 298\,K$. La pared posee un coeficiente de conductividad térmica dependiente de la temperatura $k(T) = k_0(T_0/T)^2$, con $k_0 = 0.2\,W/(m \cdot K)$. El extremo de la pared en contacto con el líquido está siempre a la temperatura T_0, mientras que en el extremo en contacto con el ambiente existe un fenómeno de convección natural de coeficiente $h = 50\,W/(m^2K)$.

Se desea hallar la distribución de temperaturas T en el interior de la pared, así como el flujo q de calor por unidad de superficie transmitido al ambiente. Se supone que la temperatura depende únicamente de una coordenada longitudinal x que recorre el interior de la pared. Para ello, se definen N nodos equiespaciados $x_1, \ldots, x_N$ a lo largo de dicha coordenada, de tal forma que $x_1 = 0$ y $x_N = L$ coinciden con los extremos de la pared en contacto con el líquido y el ambiente, véase la figura 1. Asimismo, se denota por $T_1, \ldots, T_N$ a las temperaturas en dichos nodos, es decir, $T_i = T(x_i)$. En ese caso, es posible demostrar (bajo ciertas aproximaciones que son más precisas cuanto mayor es N) que se satisface el sistema no lineal de ecuaciones

$$f_1(\mathbf{u}) := T_1 - T_0 \qquad\qquad = 0, \tag{1}$$

$$f_i(\mathbf{u}) := k_i(\mathbf{u})\frac{T_i - T_{i-1}}{\Delta x} + q = 0, \quad i = 2, \ldots, N, \tag{2}$$

$$f_{N+1}(\mathbf{u}) := -h\,(T_N - T_\infty) + q \;= 0, \tag{3}$$

donde

$$k_i(\mathbf{u}) := k\left(\frac{T_{i-1} + T_i}{2}\right) = k_0\left(\frac{2T_0}{T_{i-1} + T_i}\right)^2,$$

$\Delta x := L/(N-1)$ y $\mathbf{u} = [T_1, \ldots, T_N, q]^{\mathrm{T}}$ es un vector con las incógnitas a hallar.

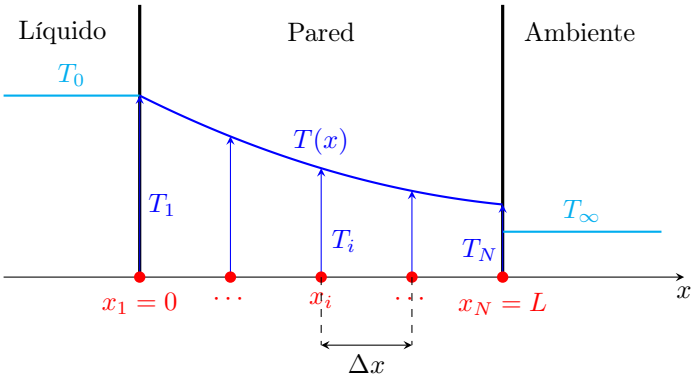

Figura 1: Esquema de la pared y de la distribución de temperatura a lo largo de la misma.

Para facilitar los cálculos, en lo que sigue se despreciarán las derivadas de $k_i(\mathbf{u})$ respecto de los elementos en $\mathbf{u}$. Es decir, a la hora de calcular derivadas, se puede tratar $k_i(\mathbf{u})$ como si fuera una constante.

Cuestión 4 (0.5pt): *Hallar los elementos no nulos de la matriz jacobiana* $\mathbb{J}_{ij}(\mathbf{u}) := \partial f_i(\mathbf{u})/\partial u_j$, *donde, como es habitual,* u_j *denota el* j*-ésimo elemento de* $\mathbf{u}$. *Utilícese una notación que deje claro qué posición ocupan dichos elementos en la matriz* $\mathbb{J}$.

Cuestión 5 (2.5pt): *Implementar una función* `[f,J] = ParedResiduo(u, CalcJ)` *que, recibido el vector* $\mathbf{u}$ *y una variable booleana* `CalcJ`, *devuelva el residuo* $\mathbf{f}(\mathbf{u}) = [f_1(\mathbf{u}), \ldots, f_{N+1}(\mathbf{u})]^{\mathrm{T}}$. *Además, si* `CalcJ` *tiene valor* `true`, *la variable* `J` *debe ser la matriz jacobiana* $\mathbb{J}(\mathbf{u})$, *en caso contrario, simplemente* `J=NaN`.

En lugar del sistema original (1)-(3), se considera el sistema precondicionado

$$\mathbf{g}(\mathbf{u}) := \frac{1}{2}\mathbb{L}^{-1}\mathbf{f}(\mathbf{u}) + \frac{1}{2}\mathbb{U}^{-1}\mathbf{f}(\mathbf{u}) = \mathbf{0}, \tag{4}$$

donde $\mathbb{L}$ y $\mathbb{U}$ son respectivamente las partes triangular inferior y superior de la matriz $\mathbb{J}(\mathbf{u}^0)$, mientras que $\mathbf{u}^0$ es la aproximación inicial, que se especificará más adelante. Se supone que $\mathbb{L}^{-1} + \mathbb{U}^{-1}$ es una matriz no singular.

En lo que sigue, puede resultar útil la función `linspace(a, b, N)`, que genera un vector fila con N elementos equiespaciados entre a y b. Asimismo, las partes triangular inferior y superior de una matriz $\mathbb{A}$ pueden extraerse respectivamente con los comandos `tril(A)` y `triu(A)`.

Cuestión 6 (1.25pt): *Implementar una función* `Problema2(N)` *que, recibido el valor de N, realice las siguientes tareas:*

- *Resolver el sistema* (4) *mediante el método de Anderson. Tómese como aproximación inicial $T^0(x_i)$ para la temperatura una distribución lineal con x_i tal que $T^0(x_1) = T_0$ y $T^0(x_N) = T_\infty$, y una aproximación inicial $q^0 = 0$ para el flujo de calor. Asimismo, tómese una tolerancia de 10^{-6}, un máximo de 100 iteraciones y 10 condiciones secantes.*
- *Mostrar por pantalla el flujo de calor q, así como la temperatura en el extremo en contacto con el ambiente, $T(x_N)$.*
- *Representar la distribución de temperaturas $T(x_i)$ frente a la coordenada x_i con una línea azul continua y con puntos (opción* `'.-b'` *del comando* `plot`*).*

Cuestión 7 (1.25pt): *Justificar la elección del residuo precondicionado* (4).

HOJA DE CONTROL

Rellenar al finalizar el examen, cuando el profesor lo indique.

Para validar el examen, copiar a continuación las líneas de código indicadas. Si la línea en cuestión está en blanco o es un comentario, escribir la primera línea posterior que contenga algún comando. Si la línea es demasiado larga, escribir únicamente entre diez y veinte caracteres (espacios no incluidos).

- Línea 25 de `NewtonModif`:
- Línea 9 de `Polin_NewtonModif`:
- Línea 25 de `ParedResiduo`:
- Línea 16 de `Problema2`: